

PLATAFORMA FOSS PARA EDUCACIÓN

Ocicat Bella

Centralizá materiales educativos y ejecutá proyectos de alumnos en contenedores aislados. Para escuelas y universidades públicas, sin licencias, auditables.

FOSS

Docker aislado

Federación entre instituciones

Auto-host

POR QUÉ EXISTE

El problema que resolvemos

“En mi máquina funciona”

El alumno sube su proyecto y en la de otro no corre: diferencias de dependencias, versiones de runtime, SO. La demo muere en el aula.

Sin presupuesto, sin licencias

Escuelas públicas no pagan SaaS. Necesitan software que corra en su propia infra, auditable y modificable por cualquiera.

Materiales esparcidos

Libros, docs y videos viven en drives personales, drives caídos, links rotos. Nada centralizado ni institucional.

Instituciones aisladas

La UTN Córdoba no comparte con la Técnica de Buenos Aires. Cada curso reinventa la rueda en su rincón.

QUÉ ENTREGA

Alcance del MVP

Materiales

Subir y listar libros, documentos y videos desde la web.

Proyectos

Registrar el proyecto del alumno y su lenguaje (py/node/go).

Ejecución aislada

Lanzar el proyecto en un contenedor con límites y sin red.

Federación (post-MVP)

Compartir metadatos de proyectos entre servidores.

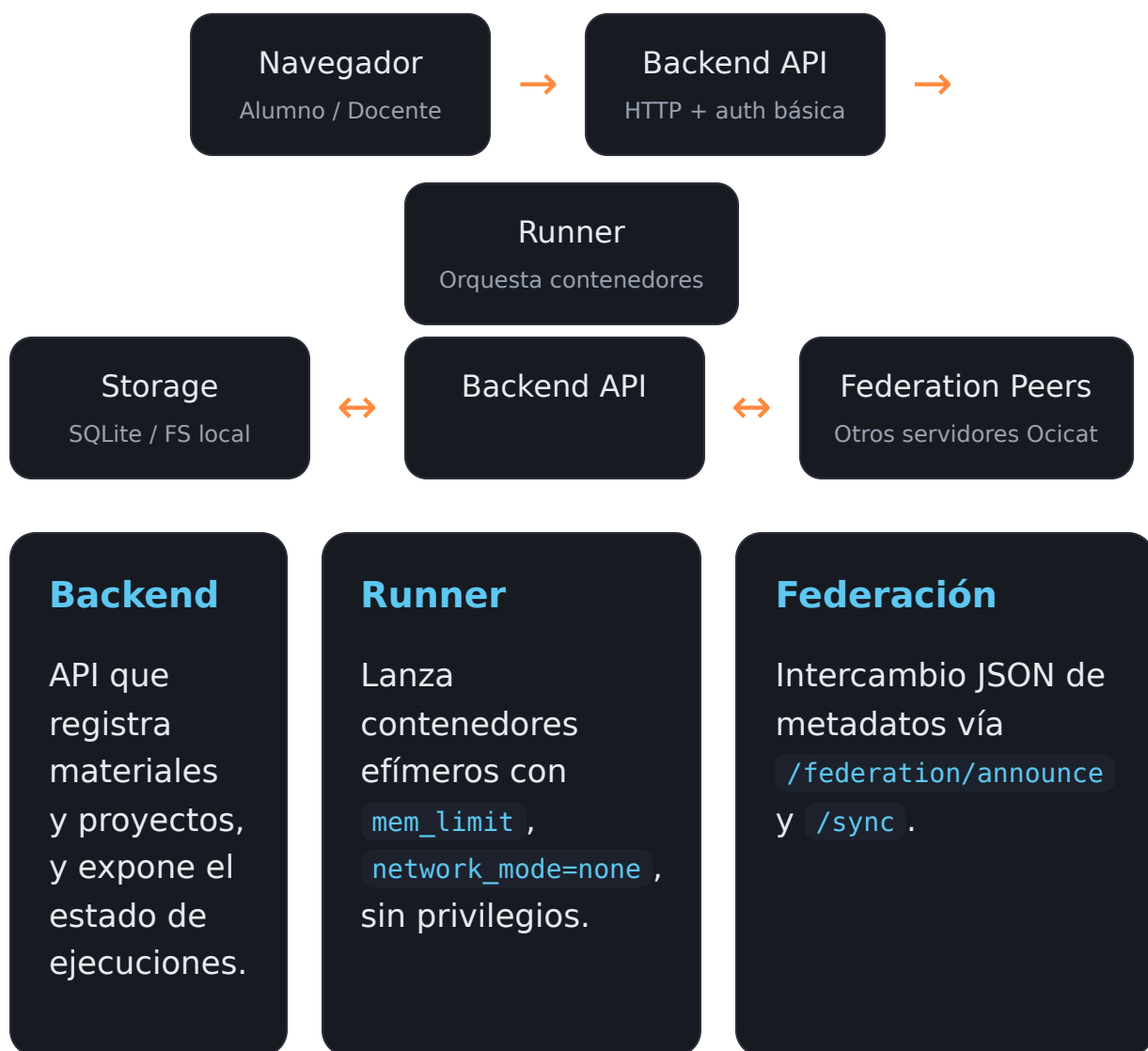
Fuentera del valor

El contenedor aislado mata el “en mi máquina funciona”: el alumno sube y corre en cualquier lado. La federación convierte una clase en una comunidad entre instituciones.

Requisito duro: FOSS y auditable. Nada de código cerrado ni telemetría.

CÓMO SE COMPONE

Arquitectura propuesta



El aislamiento es responsabilidad del Runner: sin red saliente, RAM/CPU limitadas, FS efímero.

Python → ¿Rust, TS, Go o Zig?

Lenguaje	Deploy en escuela	Mantenimiento	Runner/Docker	Veredicto
Python	Runtime (venv)	Muy alta	Medio	Actual, pero depende de runtime
Rust	Binario único	Baja (borrow chk)	Alta	Máxima seguridad, costo alto
TypeScript	Runtime (Node)	Media	Medio	Familiar, pero pesado en memoria
Go	Binario único	Alta	Excelente	Recomendado
Zig	Binario único	Muy baja	Alta	Ecosistema inmaduro

Recomendación: Go (backend + runner) + Go Templates (UI)

- **Binario único** → se copia y corre, sin versiones de runtime. Ideal para personal no experto.
- **Docker SDK de Go** es robusto y estable; el runner queda nativo.
- **Tipado estático** sin la complejidad del borrow checker de Rust.

- **UI con Go Templates** se sirve desde el mismo binario: sin SPA pesado que mantener.

La decisión final es tuya — ver “Decisiones abiertas”.

CÓMO AVANZA

Roadmap por fases

Fase 1 · MVP

- Materiales (subir/listar)
- Registro de proyectos
- Ejecución aislada básica
- SQLite + Docker

Fase 2 · Federación

- Protocolo announce/sync
- Catálogo entre peers
- Conflictos de metadatos

Fase 3 · Robustez

- UI completa
- Aislamiento fuerte (gVisor/Firecracker)
- Auth y roles
- Métricas

Cada fase entrega algo usable. No se construye todo para recién mostrar.

Aislamiento de contenedores

Contenedor = superficie de ataque

El proyecto del alumno corre código ajeno. Si escapa, compromete el host de la escuela.

Mitigaciones (MVP)

- Sin privilegios (`--user`)
- `network_mode=none` por defecto
- RAM/CPU fijas (`mem_limit`)
- Filesystem efímero

Mitigaciones (Fase 3)

- **gVisor** o **Firecracker** para micro-VM
- `seccomp` / `AppArmor`
- Cuotas de disco por proyecto

DoS

Limitar cantidad de contenedores concurrentes y recursos totales por institución.

COMUNIDAD, NO SOLO CLASE

Capa de federación

Formato

JSON con `project_id` , `name` ,
`language` , `author` .

Endpoints

POST `/federation/announce` y
GET `/federation/sync` .

Modelo

Relay centralizado simple vs P2P (decisión abierta).

Por qué importa

Un servidor en la UTN Córdoba puede anunciar un proyecto y que la Técnica de Buenos Aires lo vea en su catálogo. Los alumnos aprenden de pares de otras instituciones, no solo de su aula.

En MVP: metadatos solos. El código/binario queda en el servidor de origen.

Decisiones abiertas

1 · Lenguaje

¿Aceptás **Go** (recomendado) o querés Rust/TS/Zig? Define todo el desarrollo.

2 · UI

¿Go Templates (simple, un binario) o un SPA (React/Vue, más mantenimiento)?

3 · Aislamiento

¿Docker standard (MVP) o gVisor/Firecracker (Fase 3)?
¿Qué exige tu contexto?

4 · Federación

¿Relay centralizado (simple) o P2P (resiliente pero complejo)?

Próximo paso

Definir lenguaje → armar scaffold en Go → Fase 1 (materiales + ejecución aislada). El muse-spark-1.2-contributor de OpenCode Go queda pendiente hasta habilitar el workspace.